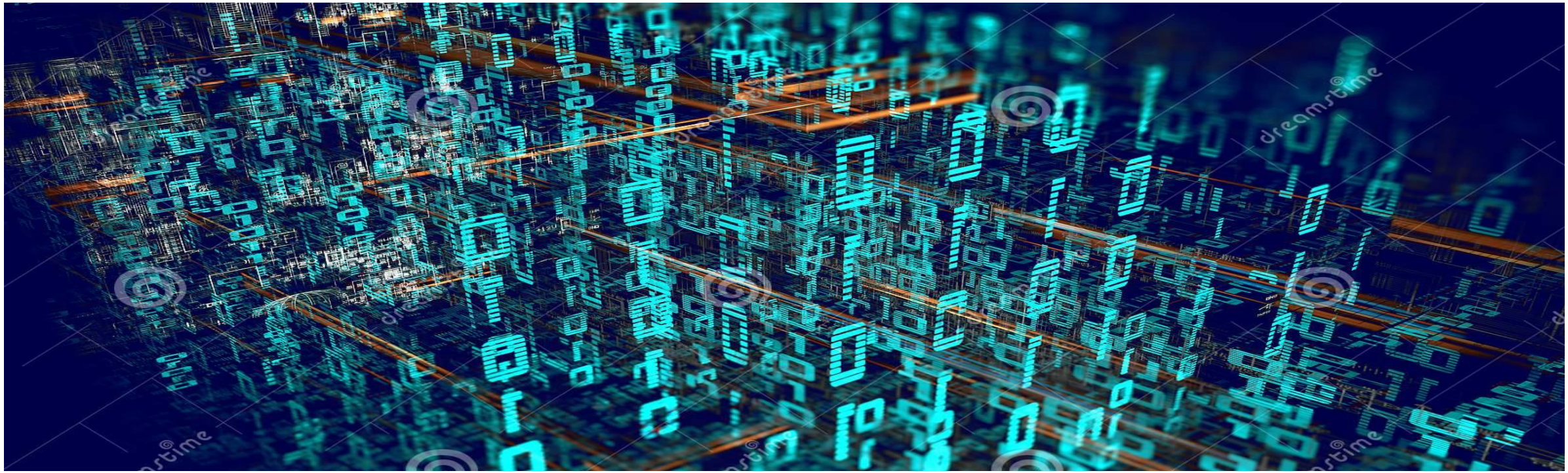
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



5

CODE OPTIMIZATION

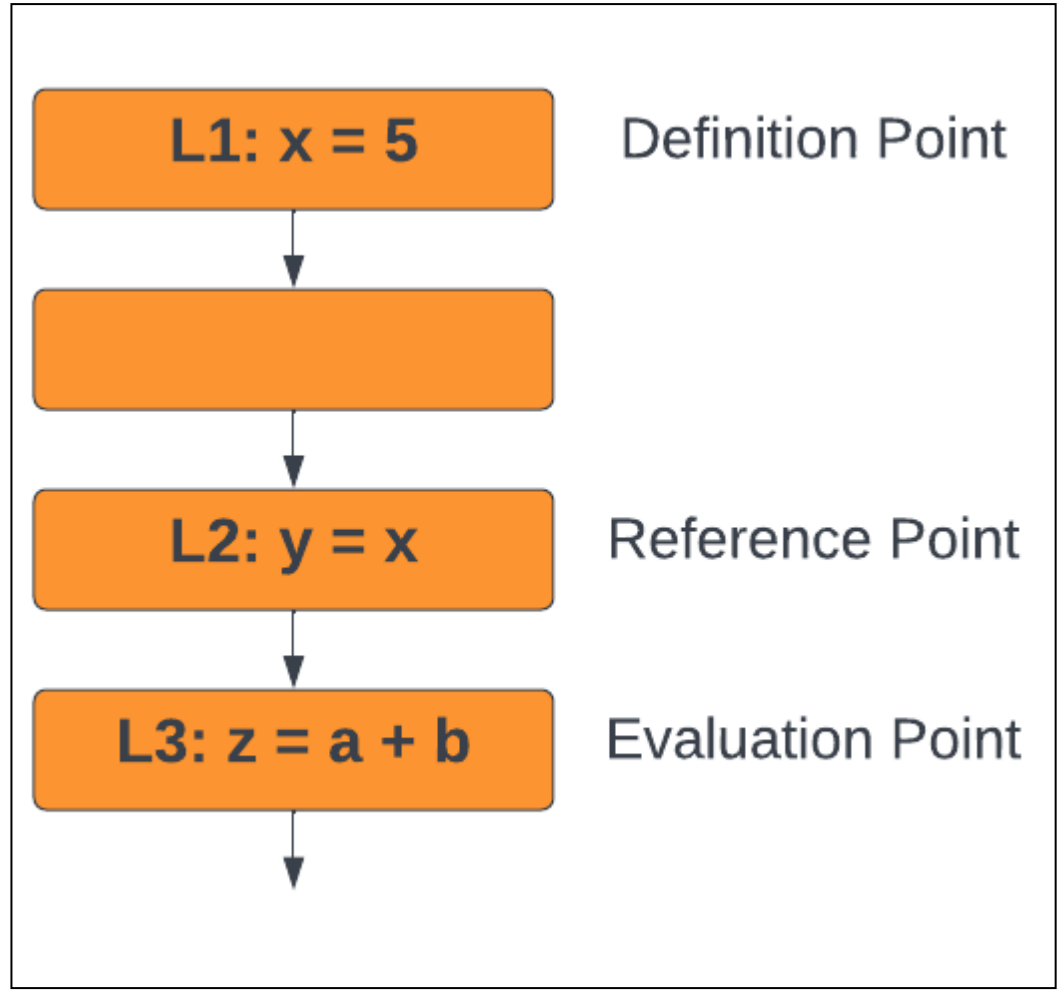
Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Introduction to Data-Flow Analysis

- "Data-flow analysis" refers to a body of techniques that derive information about the flow of data along program execution paths.
- It involves tracking the values of variables and expressions as they are computed and used throughout the program, with the goal of identifying opportunities for optimization and identifying potential errors.
- The basic idea behind data flow analysis is to model the program as a graph, where the nodes represent program statements and the edges represent data flow dependencies between the statements.
- Data flow analysis is a **global code optimization technique**.

1. Basic Terminologies

- **Definition Point**
  - A definition point is a point in a program that ***defines a data item***.
- **Reference Point**
  - A reference point is a point in a program that contains a ***reference to a data item***.
- **Evaluation Point**
  - An evaluation point is a point in a program that contains an ***expression to be evaluated***.

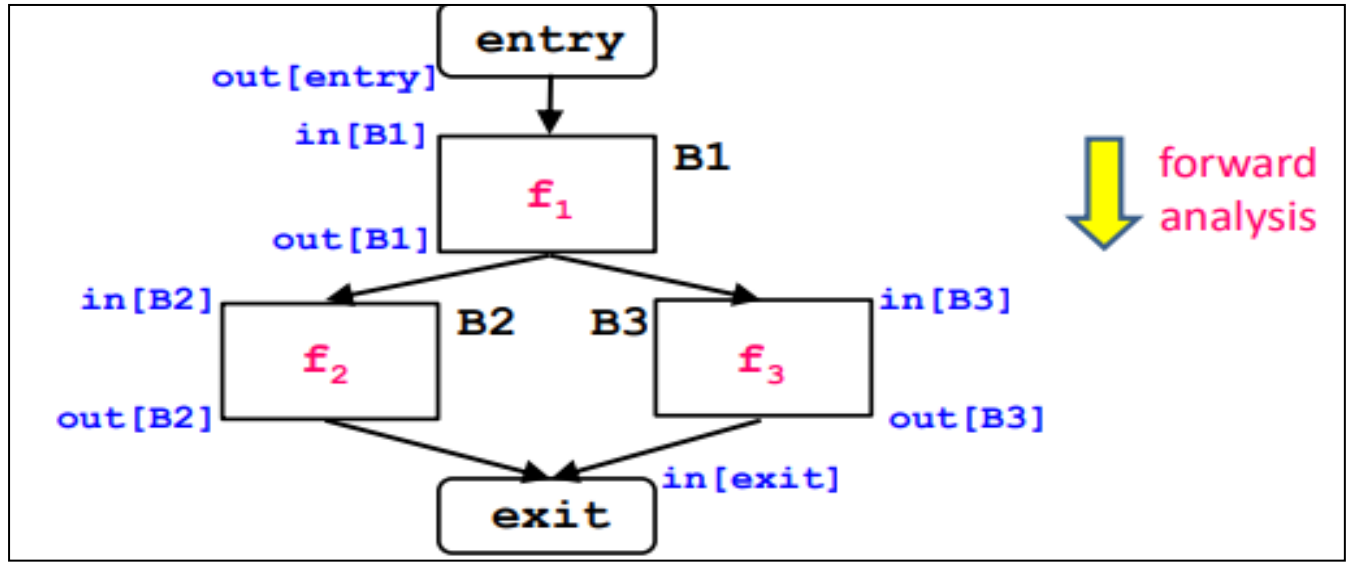


2. Data Flow Analysis Schema

- In each application of data-flow analysis, we associate with every program point a ***data-flow value*** that represents an abstraction of the set of all possible program states that can be observed for that point.
- The set of possible data-flow values is the ***domain*** for this application.
- The data-flow values before and after each statement *s is* denoted by

  IN[a]: The data flow value before the statement **a.**

  OUT[a]: The data flow value after the statement **a.**
- The main aim of the data flow problem is to ***find a set of constraints*** on the IN[a]'s and OUT[a]'s for statements a.
- There are two sets of constraints:
  - **Transfer function**
  - **Control-flow constraints**



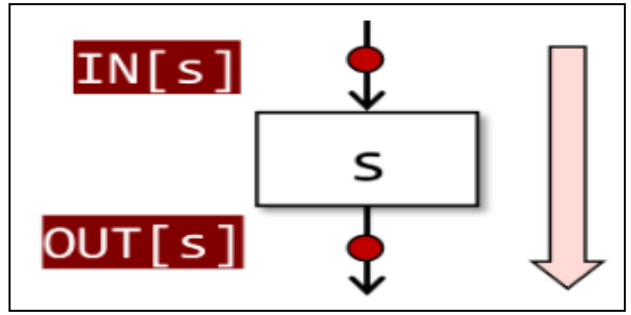
a. Transfer Function

- The semantics of the statement are the constraints for the data flow values before and after a statement.
- Example: Consider two statements.

```
x = y
z = x
```

- Both these statements are executed.
- After execution, both x and z have the same value, i.e. y.

- This relationship between the **data-flow values before** and **after** the assignment statement is known as a ***transfer function.***
- There are two types of transfer functions
  (i) Forward propagation along with execution path
  (ii) Backward propagation up the execution path



(i) Forward Propagation

- In forward propagation, transfer function is represented by **Fs** for any statement **s**.
- This transfer function accepts the data flow values **before the statement** and outputs the new data flow value after the statement.
- The new data flow or the output after the statement will be

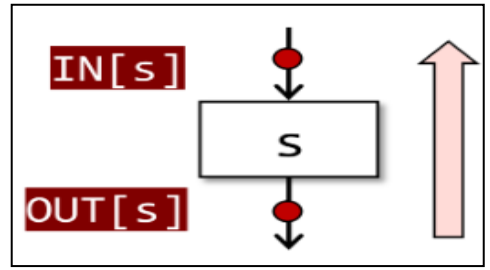
Out[s] = Fs(In[s]).x

Let's say Fs ***adds 2 to the input*** and ***.x multiplies the result by 3***.

If **In[s]** = 4, then
Fs(4) = 4+2 = 6
Out[s] = 6 * 3 = 18

(ii) Backward propagation

- Backward propagation is the reverse of forward propagation.
- This transfer function **converts a data flow value after the statement** to a new data flow value before the statement.
- The new data flow values will be

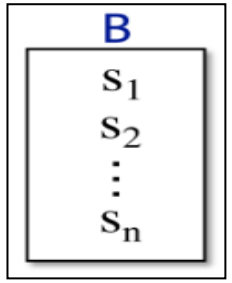


IN[s] = Fs (OUT[s])

If **OUT[s]** = 10
To find IN[s], we need to reverse Fs. Solve x + 2 = 10.
IN[s] = 10 - 2 = 8

b. Control-Flow Constraints

- The second set of constraints on data-flow values comes from the control flow.
- Relationship between the data flow values of two points that are related by program execution semantics
- The control flow value of Si will be equal to the control flow values into Si + 1 if block B contains statements S1, S2,....., Sn.



IN[Si + 1] = OUT[Si], for all i = 1 , 2, ...,n – 1.

Example: Let's use a new function for variety. Let's say Fs(x) = x + 3.

- Let's assume we have 3 steps:

  IN[S1] = 1

  OUT[S1] = Fs(IN[S1]) = 1 + 3 = 4

  IN[S2] = OUT[S1] = 4

  OUT[S2] = Fs(IN[S2]) = 4 + 3 = 7

  IN[S3] = OUT[S2] = 7

  OUT[S3] = Fs(IN[S3]) = 7 + 3 = 10

- The sequence becomes:

  IN[S1] = 1

  OUT[S1] = 4

  IN[S2] = 4

  OUT[S2] = 7

  IN[S3] = 7

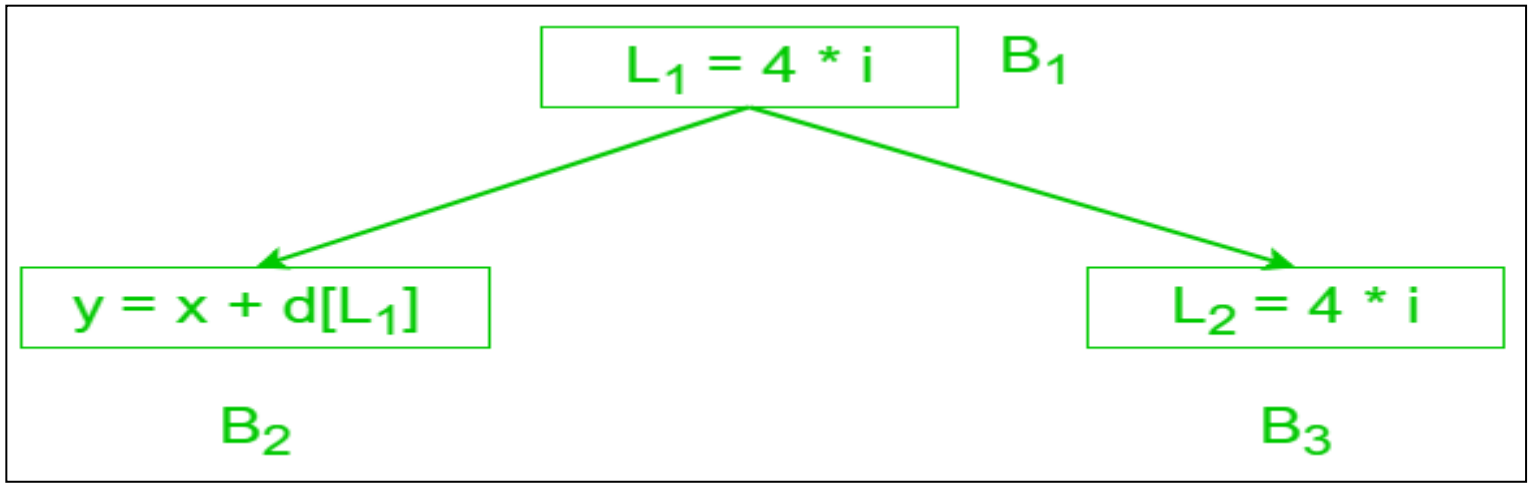
  OUT[S3] = 10

2. Data Flow Properties

- Properties of the data flow analysis are
  - ***Available expression***
  - ***Reaching definition***
  - ***Live variable***
  - ***Busy expression***

Available Expression

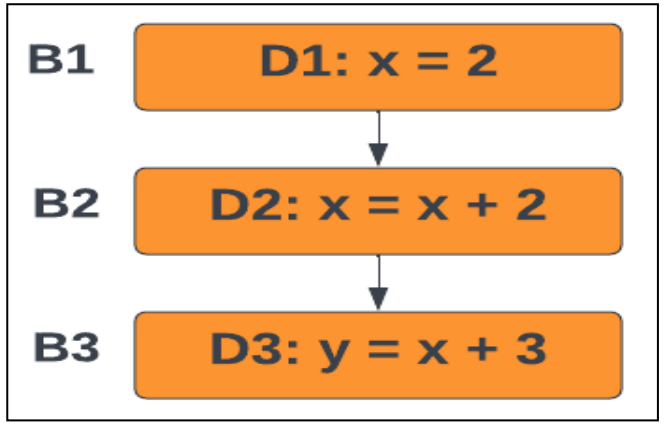
- An expression **a + b** is said to be available at a program point **x** if ***none of its operands gets modified before their use***.
- It is used to eliminate common subexpressions.
- An expression is available at its evaluation point.



L1: 4 * i is an available expression since this expression is available for blocks B2 and B3

Reaching definition

- A definition **D** is reaching a point **x** if D is ***not killed or redefined before that point***.
- It is generally used in variable/constant propagation.

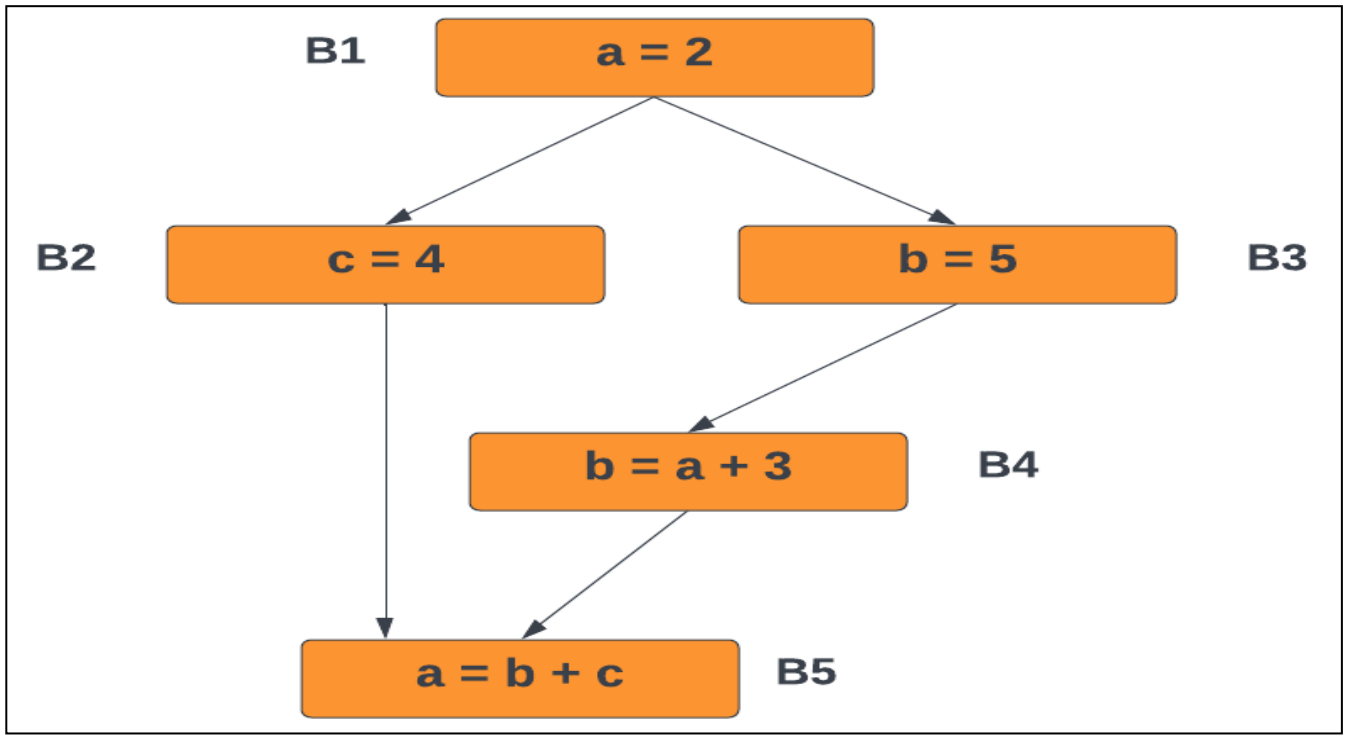


- D1 is reaching definition for B2 but not for B3, since it is killed by D2

```
1. a = 5;
2. if (condition)
3.     a = 10;
4. print(a);
```

Live variable

- A variable **x** is said to be live at a point **p** if the ***variable's value is not killed or redefined by some block***.
- Means that its value is still needed for some future computation
- If the variable is killed or redefined, it is said to be dead.
- It is generally used in register allocation and dead code elimination.



- The variable a is live at blocks B1,B2, B3 and B4 but is killed at block B5 since its value is changed from 2 to b + c.
- Variable b is live at block B3 but is killed at block B4.

Busy expression

- An expression is said to be busy along a path if its evaluation occurs along that path, but none of its operand definitions appears before it.
- It is used for performing code movement optimization.

```
1. a = b + c;
2. if (condition)
3.     x = b + c;
4. y = b + c;
```